Topic 0 switches to systems

Topic 1 Number system

Decimal Point $25.13 = 2\times10^{1} + 5\times10^{0} + 1\times10^{-1} + 3\times10^{-2}$

$(11001.1010)_2 = 1*2^4 + 1\times2^3 + 0\times2^2 + 0\times2^1 + 1\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} + 0\times2^{-4} = (25.125)_{10}$

Decimal to Binary conversion (Integer Part)

$(29)_D = (11101)_B$ ⟵

29 | 2
28 14 | 2
1 14 7 | 2
0 6 3 | 2
1 2 1
1

⟵ 11101

Decimal to Binary conversion (Fractional Part)

0.61 ⟵

0.61×2 = 1.22
0.22×2 = 0.44
0.44×2 = 0.88
0.88×2 = 1.76
0.76×2 = 1.52
0.52×2 = 1.04

$(0.67)_{10} = (.100111)_2$

Hex and Octal ↷ 8 (16 ↙ Hex)

00011101.10011100 ⟶ Hex ⟶ 1D9C
1 D 9 C

برای Octal ← ۳ تا ۳ تا جدا می کنیم

Coding system (0 - Binary)

1- Signed Numbers (Sign and Magnitude)

Sign	Magnitude

0	1001001	+73
1	1001001	-73

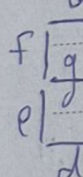
2- Binary Coded Decimal (BCD)

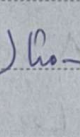
هر رقم عدد را با ۴ بیت به صورت باینری نشان می دهد

8800_55
1000 1000 0000 0000 0101 0101

3- Seven Segment Display Code (SSD code)

هر کدام از یازده خط های نمایش دیجیتالی یک حرف انگلیسی دارد که با آن ها به صورت باینری نمایش می دهیم.



مثال → 0010 →  → 1101101

4- Gray code در این سیستم هر دو عدد متوالی دقیقا در یک بیت تفاوت دارند.

جدولش تو Pdf ویدئو ها است

Subject: ...
Year. Month. Date. ( )

5- Ascii این هم یک جدول داره که در pdf است و کد اسکی دکمه های کیبورد را به باینری نشان می دهد به این طریق که یک جدول 8 × 16 موجود است و برای مثال دکمه 5 در ستون 3 و سطر 5 است پس نمایش آن به صورت ← ۰۱۰۱۱۰۱ می باشد (البته استون را باینری می کنیم نه سطر)

Binary arithmetic (جمع و تفریق در باینری)

```
  +1 +1      +1
  1 0 1 1 0 1
+ 0 1 1 0 0 1
-------------
  1 0 0 0 1 1 0
```

```
  0 1 2 2 2
  1 0 1 1 0 1
- 0 0 1 1 1 0
-------------
    1 1 1 1
```

52 - 37 = 52 - 37 + 100 - 100 ← یک روش در تفریق دسیمال که borrow ندارد

= 52 - 37 + (99 + 1) - 100 = 52 + (62 + 1) - 100 = 52 + 63 - 100

= 115 - 100 = 15

00110100 - 00100101 = ؟ حین روش برای باینری که borrow ندارد ←

00110100 - 00100101 + 10000000 - 10000000 =

00110100 - 00100101 + 11111111 + 1 - 100000000 =

00110100 + 11011010 + 1 - 100000000 = 00001111 - 100000000 = 1111

2's Complement System (مکمل ۲) بیت ها را قرینه می کنیم و 1+ می کنیم کل عدد رو

01000101 → 10111010 +1 → 10111011

روش مکمل ۲ طبق توضیحات عملیات های تفریق از جمع استفاده می کنیم

01100100 - 00101100 → مکمل ۲ → 01100100 + 11010100 = 00111000 → 00111000

```
  +1 +1      +1 +1
  0 0 0 1 1 0 1 1
- 0 1 0 0 1 1 0 1   (negative)
-------------------
  1 0 1 1 0 0 1 1
+ 1 1 0 0 1 1 1 0
-------------------
  1 1 0 0 1 1 1 0   (Binary)
```

-(00110010) = -۴۹

overflow اگر محدودیت عددهای ما ۸ بیت باشد (بیت چپ sign است) و جمع آن ها بیش از ۸ بیت شود اصطلاحاً overflow رخ داده است ←

یا جمع ۲ تا عدد مثبت یک عدد منفی شد که هم overflow رخ داده است

```
  0 1 1 0 1 0 0
+ 0 1 1 0 1 0 0 0
-----------------
1 0 0 1 1 1 0 0
```

```
  0 1 1 0 1 1 0 1
+ 0 1 1 0 0 1 0 1
-----------------
  1 1 0 1 0 0 1 0
```

جمع ۲ عدد مثبت، منفی شد

```
  1 0 0 0 0 1 1 0
+ 1 0 1 1 1 1 0 0 1
-------------------
  0 0 1 1 1 1 1 1
```

جمع ۲ عدد منفی، مثبت شد

PAPCO

Subject:

Year. Month. Date. ( )

Sign-Extension

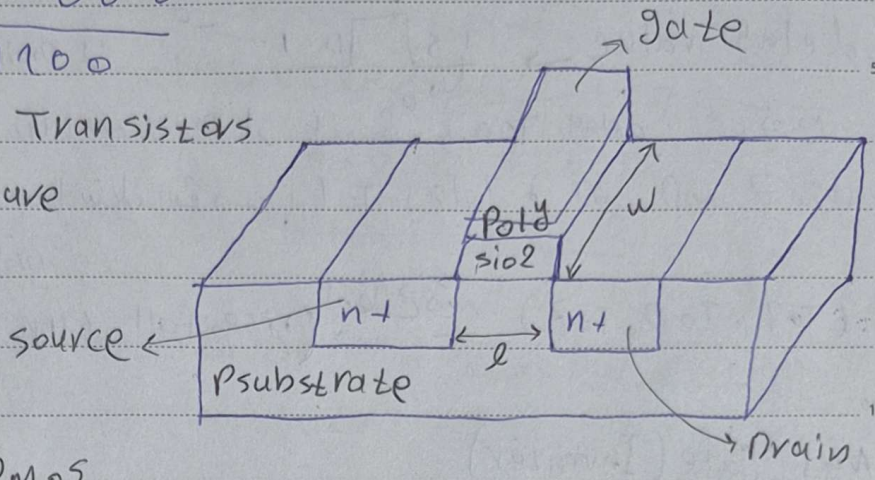
رفع مشکل overflow

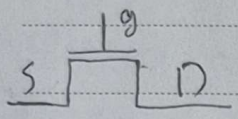
تعداد بیت ها را افزایش بدهیم و آن ها را با sign عدد پر کنیم

```
00000000 00110100
00000000 01101000
+
00000000 10011100
```

NMOS and PMOS Transistors

Transistor structure



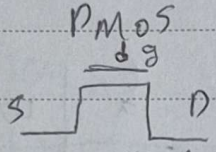


supply 0

ولتاژ مثبت conduct می کند

ولتاژ منفی conduct نمی کند

★ الکترون باعث جابجایی می شود.



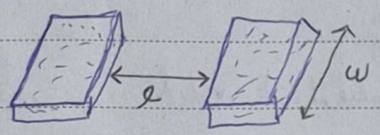
supply 1

ولتاژ منفی conduct می کند

ولتاژ مثبت conduct نمی کند

★ نبود الکترون (Hole) باعث جابجایی ولتاژ می شود.

اگر برای دو defasion های درون ترانزیستور length ، width را در نظر بگیریم:



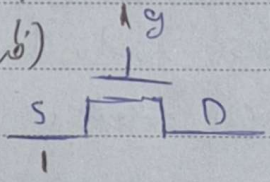
physical propeties (خواص فیزیکی): $R \sim \frac{l}{w}$

$C \sim l \times w$

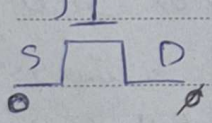
R → channel Resistance (مقاومت)

C → gate capacitance (ظرفیت)

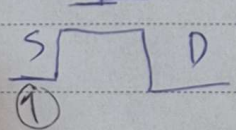
value logic:



logic one, True



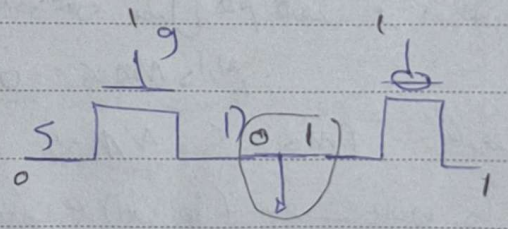
logic ★ zero, false



conduct نمی کند

Z High impedance

(Z = بی اثر)



X Unknown

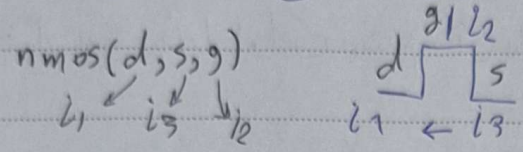
PAPCO

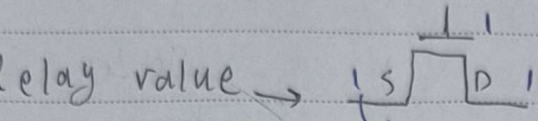
Subject:

Year. Month. Date. ( )

شبیه‌ساز مدارهای دیجیتال که پایه آن ترانسیستور است ← verilog

nmos(d, s, g)



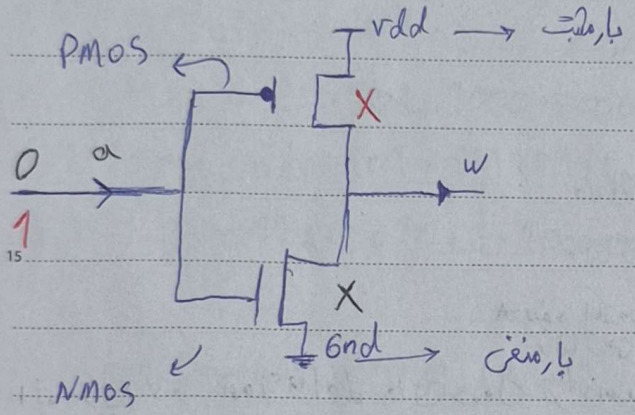


اگر Source را تغییر دهیم مقدار Drain از ۱ به ۰ تغییر می‌کند که به مدت زمان تغییر Drain از ۱ به ۰ می‌گویند delay T00

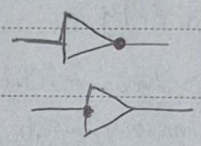
همچنین اگر با تغییر S، D به ۱ تبدیل شود delay T01 و اگر D به Z تبدیل شود delay T0Z می‌گویند.

(T01, T00, T0Z) اصطلاح دیگر (rise, fall, turn off)

NOT gate (Inverter)



a	w
0	1
1	0



نماد

$w = \neg a = \overline{a} = \sim a = NOT\,a = a'$

logical, Boolean, Verilog, VHDL, Text

توضیحات: vdd دارای بار مثبت (1) و gnd دارای بار منفی (0) است اگر a = 0 باشد ترانزیستور NMOS پایین conduct نمی‌شود یعنی عملاً انگار صفر را داریم که خروجی (1) را می‌دهد. و اگر a = 1 ترانزیستور PMOS بالایی conduct می‌شود یعنی عملاً انگار صفر را داریم که خروجی (0) را می‌دهد.

تعریف CMOS:

دو قسمت Pull up و Pull Down دارد که دو بخش مکمل هم هستند و در قسمت بالا (Pull up) PMOS داریم و در قسمت پایین NMOS داریم.

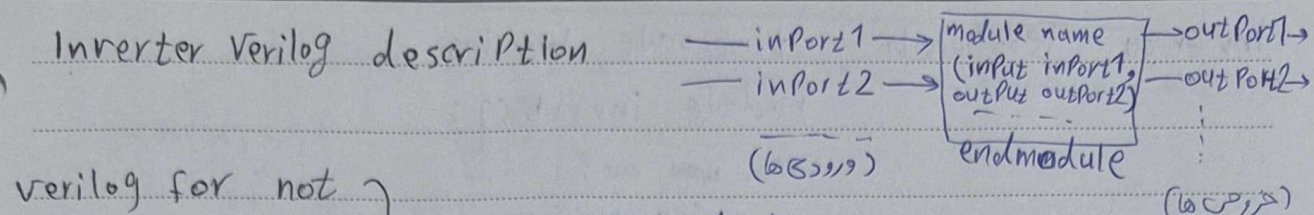
سرعت ترانزیستورها PMOS < NMOS

سرعت e (الکترون‌ها) > سرعت hole

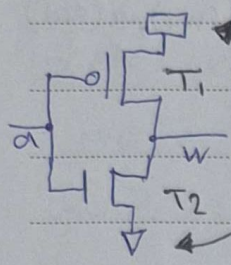
Subject: ...

Year. Month. Date. ( )

Inverter Verilog description



verilog for not

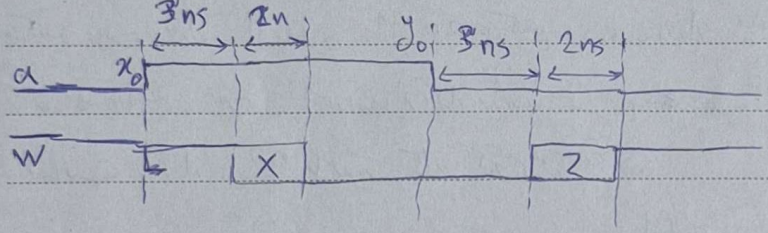



```
timescale 1ns/1ns
module class1 (input a, output w);
supply1 Vdd;
supply0 Gnd;
Pmos #5 T1(w,Vdd,a);
nmos #3 T2(w,Gnd,a);
endmodule
```

توضیحات خودمون به طور کلی چیکار کردم :

ما یک منبع (supply) 1,0 داریم که Vdd=1 و Gnd=0 هر ترانزیستور رو گفتیم با ۳ مولفه شون می دهیم برای مثال $T_1(w,Vdd,a)$ یعنی خروجی w ، ورودی a و به منبع Vdd(supply) متصل است

نحوه کار کرد a و w ←



توضیحات خودمون

در کل در این نمودار هامی آییم تغییرات وادر لحظه های تبدیل 0 به 1 و 1 به 0 بررسی می کنیم فرض می کنیم تا لحظه x_0 ، a=0 است ، در این حالت تا زمان x_0 ، w=1 است پس وقتی a=1 می شود بخاطر خود ، دیلی ترانزیستور T_2 بعد از 3 ثانیه conduct می کند یعنی در بازه (x_0, x_0+3) ، $T_1 \leftarrow 1$ ، T_2 ، T_1 conduct می کند و چون دیلی ترانزیستور T_1 ، 5 است پس در بازه (x_0+3, x_0+5) ، $T_2 \leftarrow Z$ می کند و $T_1 \leftarrow Z$ ، $T_2 \leftarrow 0$ می شود conduct می کند پس در بازه (x_0+5, y_0) ، $T_2 \leftarrow 0$ ، $T_1 \leftarrow 1$ بعد از لحظه y_0 که a تبدیل به 0 میشود طبق استدلال های بالا و دیلی ها در بازه (y_0, y_0+3) داریم $T_2 \leftarrow Z$ ، $T_1 \leftarrow Z$ ، در بازه (y_0+3, y_0+5) داریم $T_2 \leftarrow 0$ ، $T_1 \leftarrow Z$ و در بازه (y_0+5, ∞) $\leftarrow$ $T_1 \leftarrow 1$ ، $T_2 \leftarrow Z$ می شود

توی محیطی که inverter مون راست می کنیم و لحیم و سیم و ... ولت سنج اینا داریم می کوییم test و روی میزی که این کار هارا انجام می دهیم به آن میز می گوییم test bench

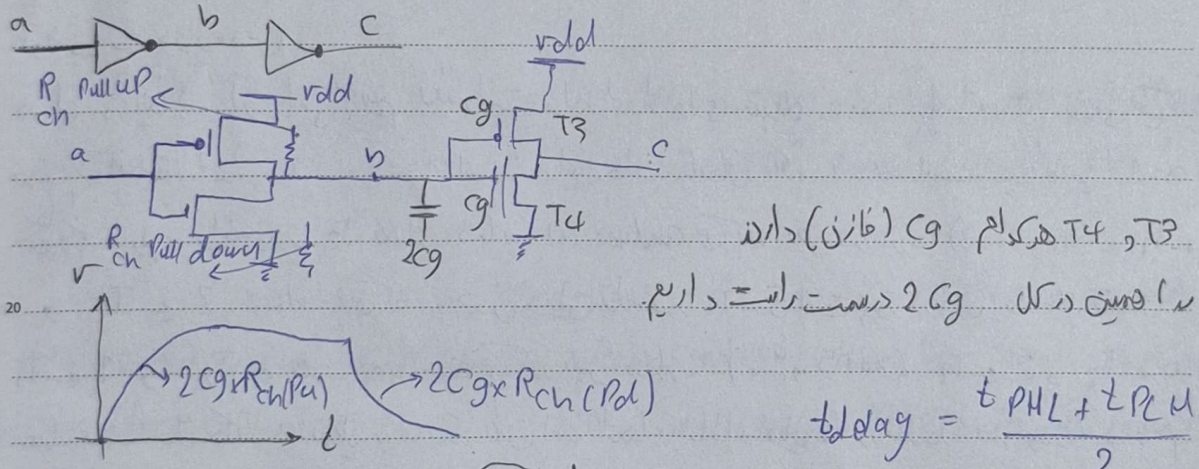
Subject: 60

Year. Month. Date. ( )

Inverter Testbench

```
time scale 1ns/1ns
module invTB();
logic aa;
wire ww;
class1 CUT1(aa,ww) t_(.w(ww),.a(aa));
initial begin
#10 aa=0;
#11 aa=1;
#11 aa=0;
#13
repeat(12) #17 aa=~aa;
#19 $stop;
ent
end module
```

توضیحات: class1 کدهای بین ساختیم را صدا می‌زنیم، ww=w, aa=a قرار می‌دهیم. بعد از initial begin دستورهایی که قرار است انجام بدهیم را می‌نویسیم. بعد از 10 ثانیه aa=0 می‌کنیم. بعد از 11 ثانیه aa=1 می‌کنیم و دوباره بعد از 11 ثانیه aa=0 می‌کنیم. بین 13 ثانیه کارهای گفته شده را 12 مرتبه این کار را تکرار می‌کنیم. هر 17 ثانیه یکبار aa=~aa می‌کنیم. در نهایت بعد از 19 ثانیه کار متوقف می‌شود.

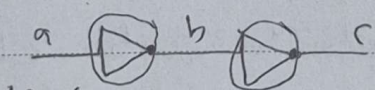


T3, T4 هرکدام Cg (خازن) دارند

و اینجا در کل 2Cg در مسیر راست داریم.

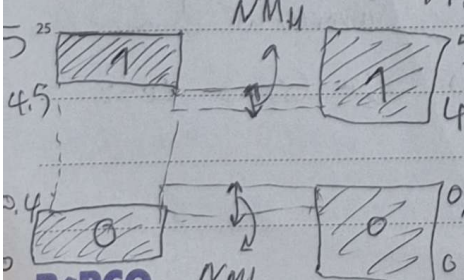
$$t_{delay} = \frac{t_{PHL} + t_{PLH}}{2}$$

noise margin:



0: 0 → 0.4
1.5 → 4.5

0: 0 → 0.6
1.5 → 4.1



$NM_H = 4.5 - 4.1 = 0.4$

$NM_L = 0.4 - 0.6 = 0.2$

$\rightarrow NM = \frac{0.4+0.2}{2} = 0.3$

PAPCO

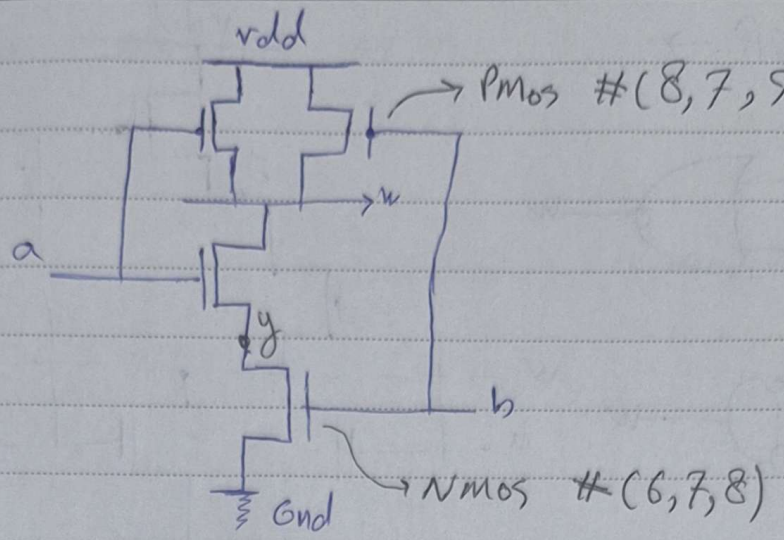
Subject: 70

Year. Month. Date. ( )

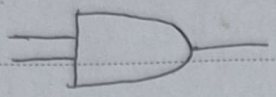
Nand gate

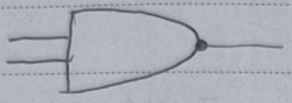
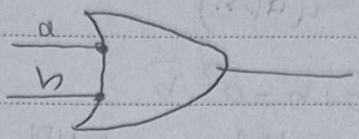
Nand = And + NOT

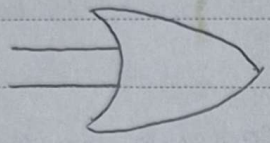
a	b	w
0	0	1
1	0	1
0	1	1
1	1	0



And =



Nand =  =  = Nand

OR = 

Nand → logic = $a \uparrow b$ verilog = $\sim(a \& b)$

boolean = $a \cdot b$

Nand gate verilog description

```
module mynand2(input a,b, output w);
    nand #(16,14)(w,a,b)
endmodule
```

جور دیگه هم می‌تونی بنویسی

ولی مقدار خطا بیشتر می‌شود

max(8, 8+8) = 16

max(9, 7+7) = 14

```
'timescale 1ns/1ns
module mynand(input a,b, output w);
    wire y;
    supply1 vdd;
    supply0 Gnd;
    Pmos #(8,7,9) T1(w,vdd,a);
    Pmos #(8,7,9) T2(w,vdd,b);
    nmos #(6,7,8) T3(y,Gnd,b);
    nmos #(6,7,8) T4(w,y,a);
end module
```

Nor gate

NOR = OR + NOT

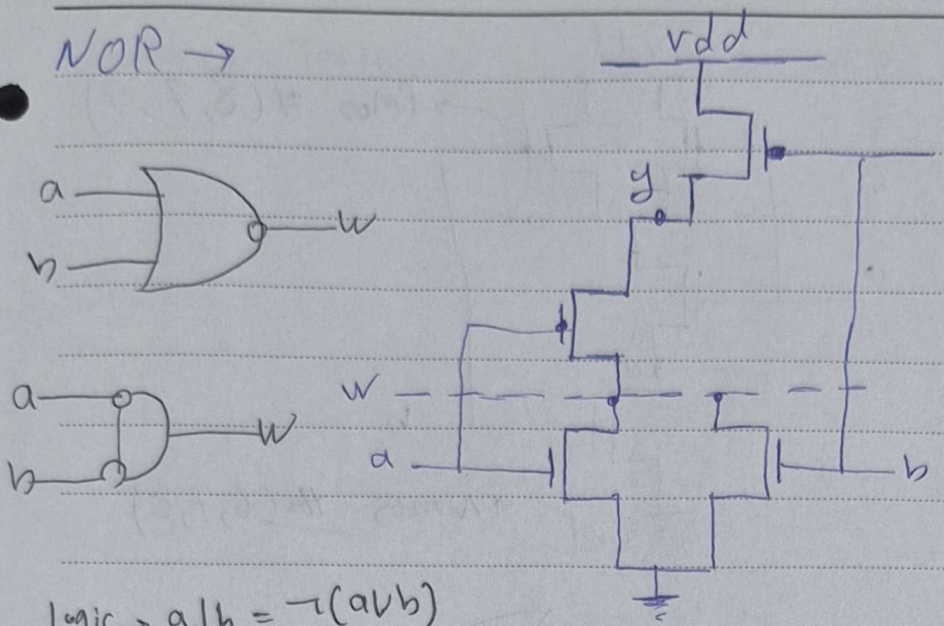
(ساختار ترانزیستوری آن برعکس NAND است فقط جای vdd و Gnd را عوض کنید)

a	b	w
0	0	1
0	1	0
1	0	0
1	1	0

Subject:

Year. Month. Date. ( )

NOR →



logic → $a \downarrow b = \neg(a \vee b)$

boolean → $\overline{a+b} = \vec{a} \cdot \bar{b}$

verilog → ~(a|b) VHDL = a NORb Text → (a+b)

NOR gate _verilog

```
module mynor(input a,b, output w)
  supply1 vdd; supply0 Gnd;
  wire y;
  nmos #7 T1(w,Gnd,a);
  nmos #7 T2(w, Gnd,b);
  Pmos #8 T3 (w,y,b);
  Pmos #8 T4 (y,vdd,a));
endmodule
```

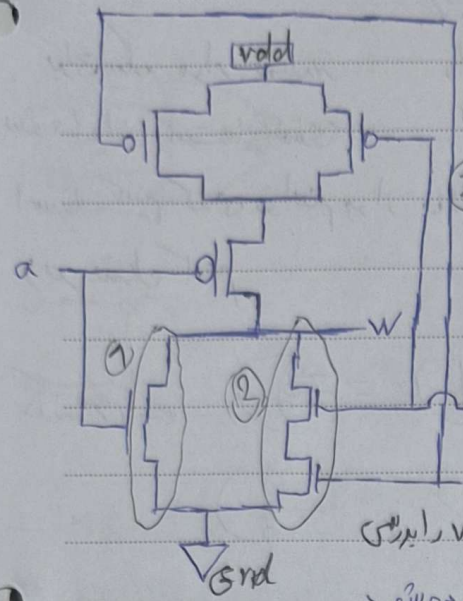
nor #(16,16)(w,a,b) ←
#16

Subject:

Year. Month. Date. ( )

Complex Gates

در این مثال تمام گیت‌ها، هم به صورت موازی به هم متصل اند هم متوالی.



(I) $w=\overline{a+b.c}$ یعنی w به ازای (a or (b & c)) صفر می‌شود

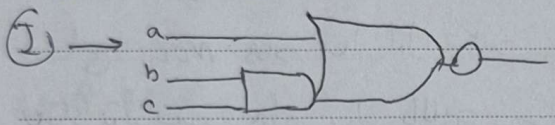
(II) $w=(\bar{b}+\bar{c}).(\bar{a})$ یعنی w به ازای $\bar{a}$ & ($\bar{b}$ | $\bar{c}$) یک می‌شود

(III) $w=(\bar{c}.\bar{a})+(\bar{b}.\bar{a})$

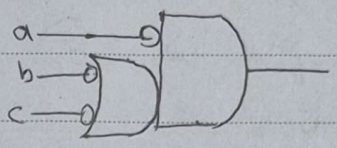
اگر $w=\bar{u}$ باشد یعنی w به ازای u صفر می‌شود و اگر $w=u$ باشد یعنی w به ازای u یک می‌شود.

حالا رابطه w را از کجا بدست میاریم؟ میاییم حالت بندی می‌کنیم مثلاً اول به خودمون می‌گیم به ازای چه a، b و c هایی w صفر می‌شود؟ بین چون w باید صفر شود می‌آییم تمام مسیرهای از w تا Grd را بررسی می‌کنیم مثلاً دوران اول 2، اداریم در مسیر 1 اگر $a=1$ باشد $w=0$ می‌شود یا اینکه در مسیر 2 هم b هم c برابر با 1 باشد بین $w=\overline{a+b.c}$ همین استدلال را برای $w=1$ هم دارد

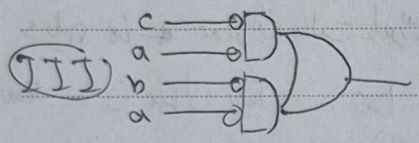
(I) →



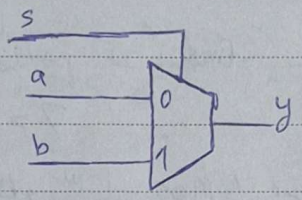
(II)



(III)



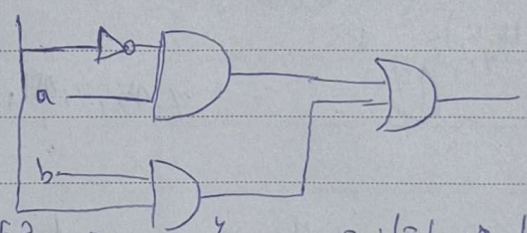
Multiplexer ←



بزبان خودمونی ← اگر $s=0$ باشد $y=a$ می‌شود و اگر $s=1$ باشد $y=b$ می‌شود.

$y=\bar{s}.a+s.b$

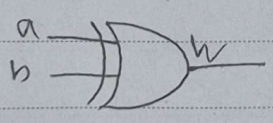
assign #40 y=s?a:b; (یک خطی if)



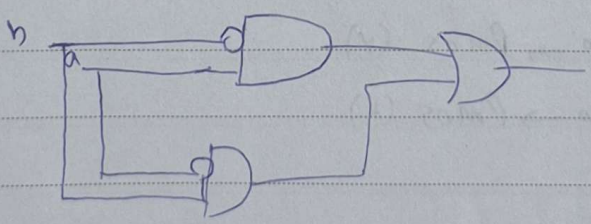
شکل مدار هم با تانسور در ویدئو 6 دقیقه 35:53 کشیده شده.

Xor gat

a	b	w
0	0	0
1	1	0
0	1	1
1	0	1



$y=\bar{a}.b+a.\bar{b}$

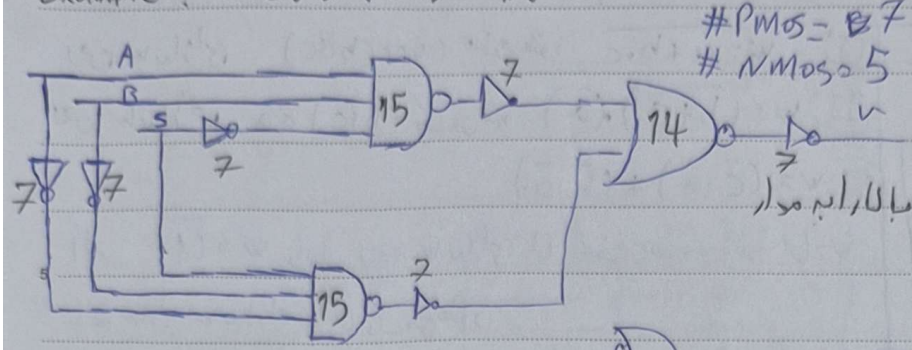


assign y = a ^ b;

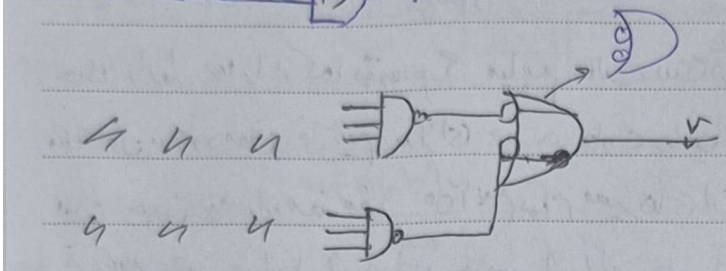
Subject: 190

Year. Month. Date. ( )

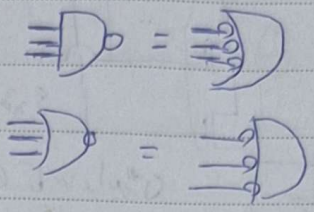
Example : overflow Detector



برای اینکه مدار بهینه‌تر شود باید از گیت‌های کمتری استفاده کنیم که می‌توانیم مدار بالا را به مدار زیر تبدیل کنیم.



نکته مهم ←



توضیحات :

هر دو مدار یکی هستند ولی مدار دوم به خاطر اینکه گیت‌های کمتری دارد بهتر است. فرقشون اینه که به جای Not های مدار اول گیت ▷ را به ▷ تبدیل کردیم.

تعداد تراز بیشتر های مدار بالا ←

دیلی ها ← بدترین حالت را بررسی می‌کنیم

مثلا برای مثال شکل ▷ را اینطور می‌کنیم که یک Pmos و یک Nmos دارد پس دیلی ← $\max(5,7) = (7)$

$mx = \begin{cases} 7 = \text{بالا} \\ 3\times 5 = \text{پایین} \end{cases}$ ← $\max(14,5) = (14)$

Nand ← ▷ تا ۳ Pmos موازی در بالا و ۳ تا Nmos متوالی در پایین

Nor ← ▷ تا ۲ Pmos متوالی در بالا و ۲ تا Nmos موازی در پایین

پس بدترین مسیر دیلی در مدار بالا این ← $7+15+7+14+7 = (50\ ns)$

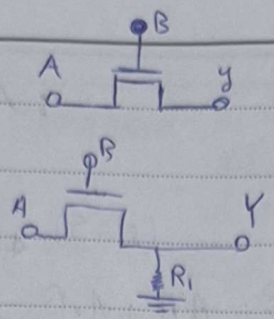
نکته مهم ← این را یاد بگیرید که ساختار Nand ، n تا موازی Pmos داره و n تا متوالی Nmos داره و Nor هم برعکس اینه.

Nand → موازی → Pmos (x)

Nor → متوالی → Pmos (+)

Subject:
Year. Month. Date. ( )

Pass Transistors:

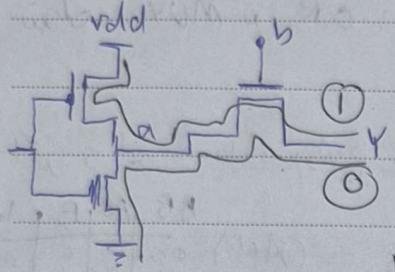


A	B	y	Y
0	0	Z	0
0	1	∅	0
1	0	Z	0
1	1	1	0

این مقاومت باعث می‌شود که Z در خروجی به عنوان ∅ interpret شود.

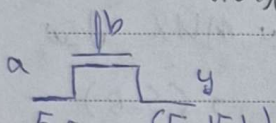
Pass transistor ها ۳ مشکل عمده دارند:

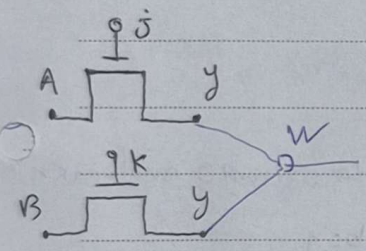
① این خروجی که Z هست در هنگام ۰ یا ۱ شدن هیچ supply ای ندارد که ولتاژ معین بگیرد پس supply دائمی از کجا می‌گیرد؟ پاسخ: از گیت های قبلی خود.



② خود این Pass transistor یک مقاومتی دارد که روی خروجی می‌گذارد.

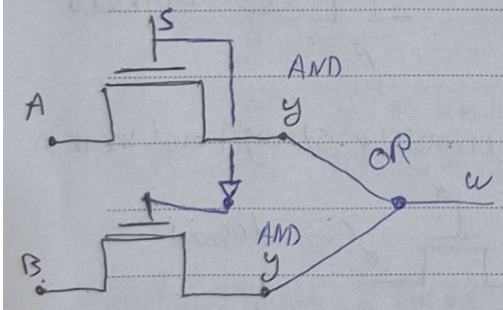
③ در اینجا NMOS ترانزیستور خود از ولتاژ اصلی یک V_{th} کم می‌کند یعنی "threshold voltage"



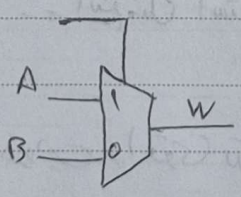


J	K	W
0	0	Z
0	1	B
1	0	A
1	1	X

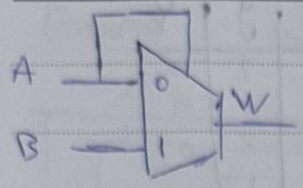
با یک inverter (Not) می‌توان یک multiplexer ساخت.



$W = \bar{S}.B + S.A \longrightarrow MUX$

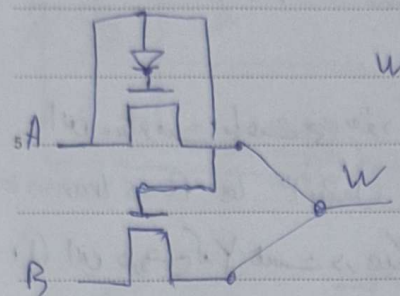


تبدیل MUX به AND:

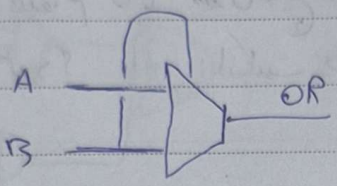


اگر S را به A وصل کنیم داریم (A=S)

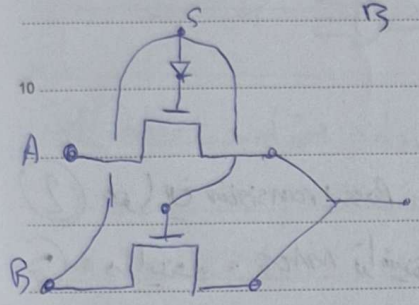
$W = \bar{S}A + S.B = \bar{A}A + A.B = \boxed{A.B}$



تبدیل MUX به OR:



اگر S را به B وصل کنیم داریم (B=S)



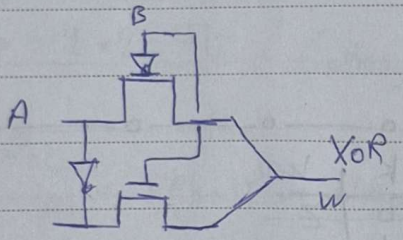
$W = A.\bar{S} + B.S = A\bar{B} + B = (A+B)$

AB	AB̄	B	W
0 0	0	0	0
0 1	0	1	1
1 0	1	0	1
1 1	0	1	1

تبدیل MUX به XOR:

$(\bar{B}=A)$ ، $(S=B)$

$W = A\bar{S} + BS = A\bar{B} + \bar{A}B = XOR$

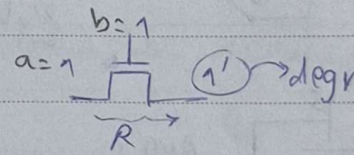


Transmission gate:

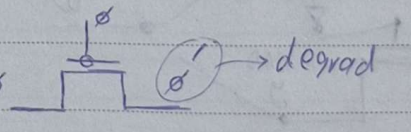
• NMOS Pass transistor: strong 0 and weak 1

قوی ← ما وقتی a و b = 1 باشد خروجی کمی از 1 کمتر است

چون مقاومت NMOS مقداری از ولتاژ را کم می‌کند.



• PMOS Pass transistor: strong 1 and weak 0



← اینهم مثل استدلال بالا اگر a و b = 0 باشد خروجی 0 نمی‌شود

حال به شرکت 200iQ می‌رویم و خروجی را قوی‌تر می‌کنیم.

Subject: 17

Year. Month. Date. ( )

کارنوف: نقشه هایی که می توانند با هم ترکیب شوند را کنار هم قرار می دهد.

2-variable map

a	b	w
0	0	0
0	1	0
1	0	0
1	1	1

truth table

b \ a	0	1
0	0	0
1	0	1

w

karnaugh map

b \ a	0	1
0	0	(1)
1	1	(1)

خوشه

$w = ab + a\bar{b} + \bar{a}b$

Product term: Any number of variables anded togheter is called a product term like → $ab, \bar{a}b, ac\bar{d}\bar{e}$

Sum of Product: (SoP): Any number of variables that ored togheter like → $a + bc + cd + \overline{dab} + \dots$ (SoP)

Minterm: A Product term with all function variables.

like→ a,b,c :ورودی → minterm: $ab\bar{c}, abc, \overline{abc}$

Standard sum of Product:

فرمی نهایی فانکشنی داشته باشیم که تمام Product term هایش minterm باشند.

physical adjacency:

اگر در karnaughmap دو خانه مجاور دو تایی 1 1 داشته باشیم می توان این دو ترم را در standard sop باهم تلفیق کرد.

implicant: Any Product term that when is 1, function is 1.

مثال: $w = ab + \bar{a}b + a\bar{b}$

b \ a	0	1
0	0	1
1	1	1

implicant: $ab, a\bar{b}, \bar{a}b, a, b$

Prime Implicant: Any Implicant that is not completly coverd by other Impleant like → a,b is above example

Essential Prime Impicant (EPI): Implicant with at least a minterm not coverd any other prime Implicant. like → a,b in above example

PAPCO

Subject:

Year. Month. Date. ( )

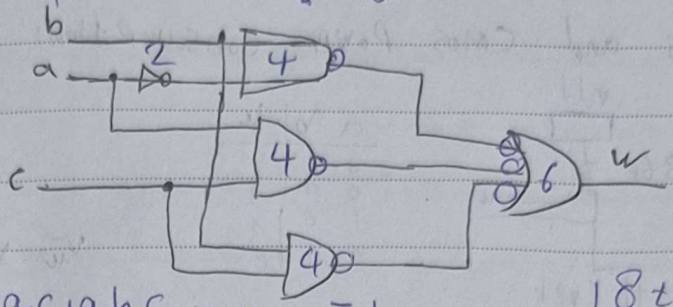
۱. گیت ایراتور ها : ① Not ② AND ③ OR

نکته: در یک معادله هرجا ایراتور هست را معکوس کنیم و هرجا عدد هست را هم معکوس کنیم معادله درست است. ← مثال

$A \cdot 1 = A \longrightarrow A \oplus 0 = A$ (کاری ندارم Not ها)

example:

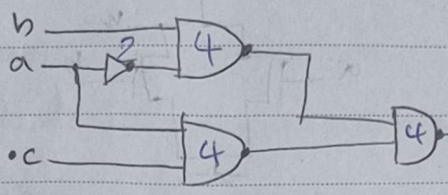
$w = a \cdot c + b \cdot c + \bar{a} \cdot b$



$w = a \cdot c + b \cdot c \cdot (a + \bar{a}) + \bar{a} \cdot b = a \cdot c + a \cdot b \cdot c + \bar{a} \cdot b \cdot c + \bar{a} \cdot b$ 18 transistor

$= a \cdot c \cdot 1 + a \cdot c \cdot b + \bar{a} \cdot b \cdot c + \bar{a} \cdot b \cdot 1 = a \cdot c \cdot (1 + b) + \bar{a} \cdot b \cdot (1 + c) = a \cdot c + \bar{a} \cdot b$

$w_{new} = \overline{(a \cdot c)} \cdot \overline{(\bar{a} \cdot b)} \longrightarrow$



14 transistor

example: ~~$w = cd(a + \bar{a}) + a\bar{c}d + \bar{a}c\bar{d}$~~ $w = cd + a\bar{c}d + \bar{a}cd + ab\bar{c} + \bar{a}b\bar{d}$

$w = cd(a + \bar{a}) + a\bar{c}d + \bar{a}cd + ab\bar{c}(d + \bar{d}) + \bar{a}b\bar{d}(c + \bar{c}) =$

$a\,cd + \bar{a}cd + a\bar{c}d + \bar{a}cd + ab\bar{c}d + ab\bar{c}\bar{d} + \bar{a}bcd + \bar{a}b\bar{c}\bar{d}$

$ad(c + \bar{c}) + \bar{a}c(d + \bar{d})$ $\qquad$ $b\bar{c}\bar{d}$

$\longrightarrow ad + \bar{a}c + ab\bar{c}d + \bar{a}bcd + b\bar{c}\bar{d} = ad(1 + b\bar{c}) + \bar{a}c(1 + bd) + b\bar{c}\bar{d} =$

$\boxed{ad + \bar{a}c + b\bar{c}\bar{d}}$

★ Super example

~~Super~~

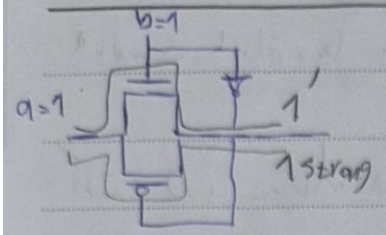
$w = \bar{a}c + \bar{a}b\bar{c} + abc =$

$\bar{a}c(1 + b) = \bar{a}c$

$\bar{a}c + \bar{a}cb + \bar{a}b\bar{c} + abc = \bar{a}c + \bar{a}cb + \bar{a}cb + \bar{a}b\bar{c} + abc = \bar{a}c + bc + \bar{a}b$

Subject:

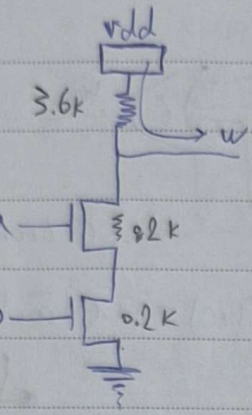
Year. Month. Date. ( )



الان دیگه degrade نشده و خوبه

الان مقاومت های N و P موازی شده اند و مقاومت معادل کمتر یا مساوی کوچکترین مقاومت میشه و بنابراین $R_{(on)}$ کاهش یافته و خروجی قوی تری میده

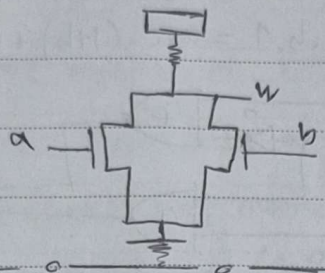
NMOS and CMOS Power consumption:



a	b	w
0	0	1
0	1	1
1	0	1
1	1	0

$$V_w = V_{dd} \times \frac{0.2+0.2}{3.6+0.2+0.2} = 0.1\,V_{dd}$$

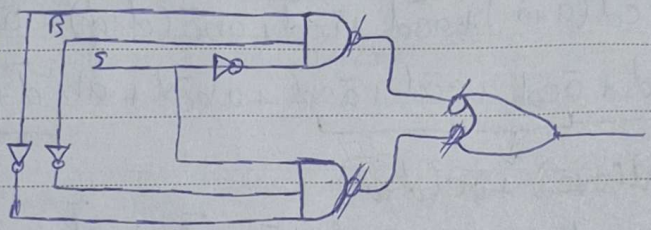
برای اینکه مشکل را حل کنیم:



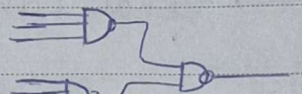
Logic Minimization topics

→ Ad-hoc method

مثال: overflow detector:



equal ||



بدون روش خاص با Not گیت ها و and و or و Nand و Nor تبدیل کنیم

Ad-hoc مشکلات:

- راه های بسیار زیاد برای تبدیل مدار
- کدام راه بهترین و بهینه ترین است؟
- آیا راه دیگری هست که delay های بهتری بدهد؟
- همواره ساده نیست Ad-hoc برای مدارهای پیچیده

boolean Algebra

(1) $A = \overline{\overline{A}}$

(2) $A+1=1$, $A.0=0$

(3) $A.1=A$, $A+0=A$

(4) $A.A=A$, $A+A=A$

(5) $A.\overline{A}=0$, $A+\overline{A}=1$

(6) $A+B=B+A$

(7) $A.B=B.A$

(8) $A.(B.C)=(A.B).C$, $A+(B+C)=(A+B)+C$

(9) $A.(B+C)=A.B+A.C$, $A+(B.C)=(A+B).(A+C)$

(10) $A+(A.B)=A$, $A.(A+B)=A$

(11) Demorgan's: $\overline{A.B}=\overline{A}+\overline{B}$, $\overline{A+B}=\overline{A}.\overline{B}$

PAPCO